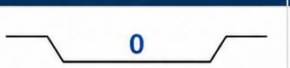
DAY2: DATA & OPERATORS

Data Types:

Verilog data types are primarily intended to represent physical data storage elements (like flip-flops) and transmission elements (like wires).

Variables and nets typically hold one of a **4-value logic system**:

- 0: Represents a logic zero, a false condition, or an electrical ground (0V).
- 1: Represents a logic one, a true condition, or the voltage supply (Vdd).
- X **or** x: Represents an unknown logic value that could be either 0 or 1. In pre-silicon verification, X-values are critical for catching reset tree bugs or uninitialized paths that cause unpredictable silicon behavior.
- Z **or** z: Represents a high-impedance state, such as an unconnected wire or a tri-state buffer

Logic Value	Symbol	Meaning	Timing Diagram Representation	Simulation Waveform (Color Convention)
0	0	Logic zero, or false condition		
1	1	Logic one, or true condition		
x	X	Unknown logic value (can be zero or one)		
z	Z	High-impedance state		

Nets:

Represent physical electrical connections between hardware entities, such as logic gates.
Net is not a keyword but represents a class of data types such as wire, wand, wor, tri, triand, trior, trireg, etc.
Here are the core characteristics of nets:

- **No Storage:** With one exception, nets do not store values on their own. Their values are continuously driven by the outputs of the devices they connect to, typically using continuous assignment (assign).
- **Default State:** An undriven net defaults to a high-impedance Z value
- **Multiple Drivers:** Unlike variables, nets can be driven by multiple sources. If standard nets receive conflicting values from multiple drivers, they resolve to an unknown state (X).
- **Synthesis:** Standard nets are fully synthesizable and translate directly into physical wires in your chip's netlist.

Standard Net Types:

- *wire*: Most common and widely used net type, functioning much like an electrical wire on a breadboard. By default, a wire is 1-bit wide, but multiple wires can be bunched together to form multi-bit vectors. If a wire is left unconnected or undriven, it naturally defaults to a high-impedance state (Z)
  wire [3:0] n0; // 4-bit wire -> this is a vector

- *tri:* The tri keyword has the exact same syntax and function as wire.

Designers use the wire keyword to denote nets that are driven by a **single driver**.

Designers use the tri keyword to explicitly indicate that the net is intended to have **multiple drivers**

Advanced Net Types

- *trireg*: Unlike all other nets, trireg models a net with inherent capacitance. It has a "driven state" where it acts like a normal wire, but if all drivers stop and go to high impedance, it enters a "capacitive state" and holds onto its last driven value.
- *tri0 and tri1:* These model resistive pull-down and pull-up devices. If nothing is actively driving the net, tri0 defaults to 0 and tri1 defaults to 1.
- *supply0 and supply1*: These are used to model the ground and power supply connections, respectively, providing a continuous, strong 0 or 1 logic value.
- *Wired Logic Nets* **(**wand**,** triand**,** wor**,** trior**)**: These nets automatically resolve logic contention when driven by multiple sources. *wand and triand* perform an implicit AND operation on all driving values. If *any* driver outputs a 0, the final resolved value of the entire net becomes 0, while *wor and trior* perform an implicit OR operation. If *any* driver outputs a 1, the final resolved value of the entire net becomes 1.
- *Unresolved nets:* unidriver wire used to model nets that allow only a single driver.
  one driver applies a 1 while another applies a 0, the conflict remains unresolved, and the net's final value becomes an unknown state (X).
  *Undriven/Unconnected Nets***:** As with an incomplete electrical circuit, if a net is not connected to anything or has no active drivers, its state is unresolved and it naturally defaults to a high-impedance value (Z)

Variables:

Variables are abstractions of data storage elements that retain their values until they are reassigned.

- **Procedural Assignment:** They are assigned procedurally and are only updated inside procedural blocks like always or initial.
- **Default State:** An uninitialized variable defaults to an unknown X value.
- **Single Driver Rule:** Variables cannot resolve multiple concurrent drivers. If multiple assignments occur, the last executed assignment wins and overwrites the previous value.

Types of Variables

- ***reg***: The most common variable is reg.
  It *does* synthesize to a flip-flop if triggered by a clock edge (e.g., always @(posedge clk)), it will synthesize into standard combinational logic (like MUXes or AND gates) if used in an unclocked block (e.g., always @(*)).
  By default, reg variables are 1-bit wide and unsigned.

- ***integer***: A 32-bit signed variable generally used for loop counters, testbench arithmetic, and transaction counting.
  integer count; // Can hold values from -2,147,483,648 to 2,147,483,647

- ***time***: A 64-bit unsigned variable used to store and track simulation time quantities for debugging.
  // time: 64-bit unsigned for storing simulation time (integer representation)
  time end_time; // Example: end_time = 50 (represents 50 time units)

- ***real*** **and** ***realtime***: 64-bit signed floating-point variables (IEEE 754 double precision). real stores standard floating-point values, while realtime stores time as a floating-point quantity.

  // real: 64-bit floating point for mathematical calculations
  real float; // Example: float = 12.344 (double-precision IEEE 754)

  // realtime: 64-bit floating point for high-precision time measurements
  realtime rtime; // Example: rtime = 40.25 (can represent fractional time units)

Types like real, realtime, and time cannot be synthesized into physical hardware and should be kept exclusively in testbench code

Example:

```
module testbench;
 integer     int_a;                          // Integer variable (32-bit signed)
 real                real_b;                         // Real variable (64-bit floating point)
 time                time_c;                         // Time variable (64-bit unsigned)

 initial begin
 int_a       = 32'hcafe_1234;      // Assign hexadecimal value to integer
 real_b      = 0.1234567;          // Assign floating point value to real

 #20;                                                // Advance simulation time by 20 units
 time_c      = $time;                        // Capture current simulation time ($time
returns time type)

 // Print all variables using $display with appropriate format specifiers
 $display ("int_a   = 0x%0h", int_a);   // %0h = hexadecimal format
 $display ("real_b          = %0.5f", real_b);  // %0.5f = 5 decimal places
 $display ("time_c          = %0t", time_c);    // %0t = time format
 end
endmodule
```

Simulation vs. Synthesis It is highly recommended to explicitly use reg (or sized vectors of reg) for all of your synthesizable hardware design (RTL). Types like integer, real, and time should be reserved exclusively for simulation testbenches.
Synthesis tools will often reject real and time variables entirely because physical FPGAs and ASICs do not have built-in floating-point hardware unless you explicitly design it. Similarly, using a full 32-bit integer in your hardware design when you only need to count to 10 will

waste significant chip area, so you should explicitly declare a smaller variable (e.g., reg [3:0]) instead.

Data Type/Net Name	Category	Bit Width/Range	Signedness	Description
wire	Net	User-defined (default 1-bit)	Unsigned	Used to connect hardware entities; does not store values. Default value is Z (high-impedance).
reg	Variable	User-defined (default 1-bit)	Unsigned	Abstraction of a data storage element; retains value until reassigned. Default value is X.
integer	Variable	32 bits	Signed	General purpose variable for loop counters and arithmetic operations.
supply0	Net	Scalar or vector	Unsigned	Driving net type used to model ground (Vss); strongest driving strength.
supply1	Net	Scalar or vector	Unsigned	Driving net type used to model power supply (Vdd); strongest driving strength.
tri	Net	Scalar or vector	Unsigned	Identical syntax to wire; used to indicate nets with multiple drivers.
trireg	Net	Scalar or vector	Unsigned	Models nets with capacitance that store values; default strength is medium.
tri0	Net	Scalar or vector	Unsigned	Models resistive pulldown devices; has value 0 if nothing is driving; default strength is pull.
tri1	Net	Scalar or vector	Unsigned	Models resistive pullup devices; has value 1 if nothing is driving; default strength is pull.
wand	Net	Scalar or vector	Unsigned	Wired-AND net; resolves multiple driver logic by performing AND operation.
wor	Net	Scalar or vector	Unsigned	Wired-OR net; resolves multiple driver logic by performing OR operation.
time	Variable	64 bits	Unsigned	Used to store simulation time quantities for debugging.
real	Variable	64 bits (IEEE 754 double)	Signed floating point	Stores floating point values; primarily for simulation only.
realtime	Variable	64 bits (IEEE 754 double)	Signed floating point	Stores time as a floating point quantity for high-precision measurements.

Strings:

A string is a sequence of one-byte ASCII characters enclosed in double quotes.
One strict syntactic rule is that a string must be contained entirely on a single line and cannot span multiple lines.
If the variable size is smaller than the string width, Verilog silently truncates the leftmost characters, keeping only the rightmost characters that fit.
If the variable size is larger than the string, Verilog automatically pads the extra space on the left side of the string with zeros (which display as empty spaces).
You can also use escaped characters within the string, such as \n to print a new line or \t to insert a tab space.

// String storage: Each ASCII character requires 8 bits (1 byte)
// "Hello World" = 11 characters = 11 bytes = 88 bits

// Exact fit: 11 characters * 8 bits = 88 bits [8*11:1]
*reg [8*11:1] str = "Hello World";* // Stores full string: "Hello World"

// Too small: Only 5 bytes (40 bits), leftmost 6 bytes truncated
*reg [8*5:1] str = "Hello World*"; // Stores rightmost 5 chars: "World"

// Too large: 20 bytes (160 bits), leftmost bits padded with zeros (spaces)
*reg [8*20:1] str = "Hello World";* // Stores " Hello World" (9 leading spaces)

Escaped Characters	Character Displayed
\n	newline
\t	tab
%%	%
\\	\
\"	"
\ooo	Character written in 1–3 octal digits

Scalar & Vectors:

- **Scalars** are single, 1-bit wide elements. By default, when you declare a data type like a wire or reg without explicitly defining its size, Verilog creates it as a 1-bit scalar.

- **Vectors** are multi-bit entities created by bunching multiple nets or variables together. They are used to model data buses that can transmit or store multiple separate values simultaneously. For example, you can declare a 4-bit wire vector that carries four independent signals on a single bus.

Hardware Synthesis: Both scalar and vector forms of wire and reg data types are fully synthesizable and translate directly into actual physical hardware logic and routing.

Path Delays and Conditions: When specifying timing and conditional path delays, your operands can freely be either scalar or vector input/inout ports, nets, or registers.

User-Defined Primitives (UDPs): There are strict limitations regarding vectors when designing custom leaf-level primitives. Verilog rules dictate that UDPs are **only permitted to have scalar (1-bit) input and output terminals**; vectors cannot be used in UDP definitions.

Bits Select: Used to access a single, specific bit out of a multi-bit vector (such as a wire or reg bus).

appending square brackets containing the index of the desired bit to the end of a variable's name (e.g., variable_name[index]).

reg [7:0] addr; // 8-bit reg variable [7, 6, 5, 4, 3, 2, 1, 0]

addr [0] = 1; // assign 1 to bit 0 of addr

addr [3] = 0; // assign 0 to bit 3 of addr

Part Select: Allows you to extract a contiguous range of multiple bits from a larger multi-bit vector (such as a wire or reg bus). By appending square brackets to the end of a variable's name and using a colon (:) to separate the upper and lower index boundaries (e.g., variable_name[msb:lsb]). Two types of part-selects, one with a constant part-select and another with an indexed part-select.

reg [31:0] addr;

addr [23:16] = 8'h23; // bits 23 to 16 will be replaced by the new value 'h23 -> constant part-select.

Category	Examples	Purpose	Default Value
Nets	wire, tri, wand, wor	Physical connections	z (high-Z)
Variables	reg, integer, real, time	Store values in procedural blocks	x (unknown)
Scalars	wire a; reg b;	Single-bit signals	Depends on type
Vectors	wire [7:0] data;	Multi-bit buses	Depends on type
Constants	parameter, localparam	Fixed values	User-defined
Arrays	reg [3:0] mem[0:7];	Indexed storage	Depends on type

Verilog Operators:

1. Verilog Arithmetic Operators:

Arithmetic operators are used to perform mathematical calculations and include multiply (*), divide (/), add (+), subtract (-), modulus (%), as well as unary plus (+) and unary minus (-) .

They can act as binary operators (taking two operands) or unary operators (specifying positive + or negative - signs)
The power operator (**) is for simulation only and does not synthesize.
Data Types and Arithmetic
Testbenches: In simulation, the 32-bit signed integer data type is the most common choice for arithmetic operations and loop counters.

If you require signed arithmetic in your hardware, the best practice is to explicitly declare a sized, signed register, such as reg signed [15:0].

```
module des;
 reg [7:0] data1;
 reg [7:0] data2;

 initial begin
 data1 = 45; // 8-bit value
 data2 = 9; // 8-bit value

 $display ("Add + = %d", data1 + data2); // 45 + 9 = 54
 $display ("Sub - = %d", data1 - data2); // 45 - 9 = 36
 $display ("Mul * = %d", data1 * data2); // 45 * 9 = 405, truncated to 8 bits = 149
 $display ("Div / = %d", data1 / data2); // 45 / 9 = 5 (integer division)
 $display ("Mod %% = %d", data1 % data2); // 45 % 9 = 0 (no remainder)
 $display ("Pow ** = %d", data2 ** 2); // 9^2 = 81 (simulation only)
 end
endmodule
```

Programming Language Interface (PLI) If you are writing user-defined C routines to interface with your Verilog simulation (PLI), Verilog provides specific utility routines for handling long arithmetic (64-bit values) . Routines such as tf_add_long(), tf_subtract_long(), tf_multiply_long(), and tf_divide_long() allow you to pass and compute large integer values across the Verilog/C boundary.

Binary Arithmetic Operators:

Binary operators compute mathematical results between two operands and must be placed directly between them.

- Addition (+) and Subtraction (-): These map directly to physical digital circuits, synthesizing into adders (e.g., ripple-carry or carry-lookahead) and subtractors (which typically use adders and two's complement representation).
- Multiplication (*), Division (/), and Modulus (%): These operators synthesize into complex multiplier circuits (like Wallace trees) and iterative divider circuits. They are highly expensive in terms of physical logic gates and silicon area. Note that integer division truncates any fractional parts toward zero, and the modulus remainder will take the mathematical sign of the first operand.
- Power (**): Raises the first operand to the power of the second, but is generally not synthesizable

Unary Arithmetic Operators:

In Verilog, unary arithmetic operators operate on a single operand to specify or change its mathematical sign.
The two unary arithmetic operators provided by the language are:

- Unary Plus (+): Explicitly specifies a positive value.
- Unary Minus (-): Negates the value of the operand, mathematically representing it as a negative number.

Syntax and Rules

- Positioning: Because they are unary operators, the + or - symbol must always appear directly to the left of their single operand.
- To explicitly handle signedness, use $signed() and $unsigned() system functions
- Two's Complement: When the unary minus is applied, Verilog internally stores and evaluates the resulting negative number using two's complement representation.
- Literal Number Declarations: If you are using the unary minus to declare a hardcoded negative number, the minus sign must be placed before the size specification of the number (for example, -6'd3).

In Verilog, the bit lengths and sizing of values within expressions must be managed carefully to avoid unintended hardware consequences:

- **X and Z States:** For any arithmetic operator, if even a single bit within an operand is an unknown (x) or high-impedance (z) value, the entire result evaluates to x.
- **Division by Zero:** If the divisor (second operand) in a division or modulus operation is zero, the result will evaluate to x. Because an x state is difficult to manage in physical hardware, designers must manually implement conditional logic safeguards to prevent hardware division by zero.
- **Synthesizability Restrictions:** All binary and unary arithmetic operators become completely non-synthesizable if applied to real or realtime floating-point data types. Standard synthesis tools only infer logic when these operators are applied to discrete binary values like reg or integer.
- **Bit-Widths and Silent Truncation:** The bit length of a binary arithmetic result is typically equal to the maximum bit length of the operands. If an addition causes a carry overflow and is assigned back to a register of the same size, the most significant bit is silently truncated and lost

2. Verilog Relational Operators:

In Verilog, relational operators are used to compare the values of two operands.
They return a 1-bit logical value: 1 or 0, but will yield an x if any unknown or high-impedance bits are present in the operands

The standard relational operators defined in Verilog are:
> : Greater than
< : Less than
>= : Greater than or equal
<= : Less than or equal

```
assign A_gt_B = (A > B); // Synthesizes logic to evaluate if A is greater than B
assign A_lt_B = (A < B); // Synthesizes logic to evaluate if A is less than B
```

3. Verilog Equality Operators:

Verilog provides two distinct categories of equality operators:

- logical equality
- case equality

Both are binary operators (taking two operands), but they have a critical difference in how they handle unknown (x) and high-impedance (z) values.
1. Logical Equality (== and !=):

check whether two operands are equal (==) or not equal (!=).
Handling of x and z: If either operand contains an x or z bit, the result of the entire comparison becomes ambiguous and evaluates to an unknown x
can return a 0 (false), 1 (true), or x (unknown)

2. Case Equality (=== and !==):

These operators perform a strict, bit-by-bit comparison of the two operands, and they include exact matches of x and z values
Handling of x and z: Treat x and z as literal values to be matched. For example, the expression 4'b1xxz === 4'b1xxz will evaluate to 1 (true) because every bit matches exactly.
It will never results in an x; it will only ever return a definitive 1 (match) or 0 (no match)
if the two operands are of unequal lengths, Verilog will automatically zero-fill the shorter operand so that its length matches the longer operand before completing the comparison.

4. Verilog Logical Operators:

Logical operators always evaluate to a 1-bit value: 0 (false), 1 (true), or x (ambiguous).
The three standard logical operators are:
! : Logical negation (NOT)
&& : Logical AND
|| : Logical OR
When an operand is evaluated, any non-zero value is treated as a logical 1 (true).
If an operand is not equal to zero, it is treated as a logical 1; if any operand bit is x or z (high-impedance), the condition evaluates to x
Most frequently used to drive control flow and conditional execution.
For example:
Procedural Blocks: They are heavily used inside if-else or while loop conditions to dictate the behavior of sequential or combinational logic.
Conditional Path Delays: When specifying precise timing models in a specify block, you can use logical operators inside if statements to assign state-dependent path delays (e.g., conditionally assigning a 9-unit delay versus a 13-unit delay depending on the current logical state of two signals).

5. Verilog Bitwise Operators:

Unlike logical operators that return a single bit, bitwise operators perform the operation on a corresponding bit-by-bit basis between two operands.

~ : Bitwise negation (a unary operator)
& : Bitwise AND
| : Bitwise OR
^ : Bitwise XOR (Exclusive OR)

^~ or ~^ : Bitwise XNOR (Exclusive NOR)

If one operand is shorter than the other, it is automatically zero-extended to match the length of the longer operand.

6. Verilog Shift Operators:

Shift operators are used to move the bits of a vector operand either to the right or to the left.

The two standard shift operators are:
>> (Right Shift): Shifts the bits of the vector to the right.
<< (Left Shift): Shifts the bits of the vector to the left.

The vacant bit positions are filled with zeros, and the operations do not wrap around.

Conditional Operator (? :)

Taking three operands, this operates exactly like an if-else statement or a hardware multiplexer.
The syntax is *condition_expr ? true_expr : false_expr*
If the condition is true, it evaluates the first expression; if false, it evaluates the second.

Verilog Concatenation:

Used to group together multiple single-bit scalars, multi-bit vectors, or sized constants to form a larger, consolidated multi-bit bus or vector, by enclosing the desired operands within curly braces { } and separating them with commas.

Syntax and Core Rules:

- Strict Sizing: Every single operand inside the concatenation expression must have an explicitly known bit-width so the compiler can calculate the total resulting size. You cannot use unsized literal numbers (for example, {a, 5} is illegal); they must be explicitly sized (like {a, 3'b101}) to avoid unpredictable results or compilation failures.
- Bit Ordering: The concatenation operator fills the resulting bus from left to right. This means the very first signal listed inside the braces will occupy the most significant bits (MSB) of the new vector.
- Left-Hand Side (LHS) Usage: Concatenation is uniquely allowed on the left-hand side of an assignment to split a bus or capture an overflow/carry bit (e.g., {carry, sum} = a + b;). When doing this, you must ensure the total bit-width on the LHS perfectly matches the width of the right-hand side expression to avoid unexpectedly dropping bits.

Verilog / SystemVerilog Operator Precedence Table (High to Low)

Precedence (High to Low)	Category	Operators	Description	Associativity	Notes / Example
1	Unary	+ - ! ~ & ~& | ~| ^ ~^ ^~	Unary plus, unary minus, logical NOT, bitwise NOT, reduction AND, reduction NAND, reduction OR, reduction NOR, reduction XOR, reduction XNOR	Right to Left	~a, !b, &bus, ~&bus, ^data
2	Power	**	Exponentiation	Right to Left	a ** b
3	Multiply / Divide / Modulus	* / %	Multiplication, division, modulus	Left to Right	a * b / c % d
4	Add / Subtract	+ -	Addition, subtraction	Left to Right	a + b - c
5	Shift	<< >> <<< >>>	Logical left, logical right, arithmetic left, arithmetic right shift	Left to Right	a << 2, b >>> 1
6	Relational	< <= > >=	Less than, less or equal, greater than, greater or equal	Left to Right	a < b, c >= d
7	Equality	== != === !==	Equality, inequality, case equality, case inequality	Left to Right	a == b, c !== d
8	Bitwise AND	&	Bitwise AND	Left to Right	a & b
9	Bitwise XOR	^	Bitwise XOR	Left to Right	a ^ b
10	Bitwise XNOR	^~ ~^	Bitwise XNOR	Left to Right	a ^~ b
11	Bitwise OR	|	Bitwise OR	Left to Right	a | b
12	Logical AND	&&	Logical AND	Left to Right	a && b
13	Logical OR	||	Logical OR	Left to Right	a || b
14	Conditional (Ternary)	? :	Conditional operator	Right to Left	sel ? a : b
15	Concatenation	{ }	Concatenation	Left to Right	{a, b, c}
16	Replication	{ {} }	Replication	Left to Right	{4{a}}
17	Streaming (SystemVerilog)	<< >> {<< >>} {>> <<}	Streaming concatenation operators	Left to Right	{<<8{a}}, {>>{b}}
18	Assignment	= += -= *= /= %= &= |= ^= <<= >>= <<<= >>>= <<= >>>=	Assignment and compound assignment operators	Right to Left	a = b, a += 1, b <<= 2
19	Comma (SystemVerilog)	,	Comma operator (sequence of expressions)	Left to Right	f(a, b, c)

Notes: 1. When two operators have the same precedence, the associativity determines the order of evaluation.
2. Parentheses ( ) can be used to override the default precedence.

Replication Operator:

A special form of concatenation used to repeat a specific expression a set number of times to easily generate a larger, multi-bit pattern.
Syntax and Structure:
It is defined by placing a multiplier constant and the target expression inside an additional set of braces: {count{expression}}
For example, {7{a}} repeats the 1-bit signal a seven times, creating a 7-bit vector.
Rules and Limitations

- Multiplier Rules: The replication constant (count) must be a non-negative number. It is strictly illegal to use a dynamically changing variable, an X, or a Z as the replication constant.
- **LHS Restrictions:** Replication expressions cannot appear on the left-hand side (LHS) of any assignment, and they are not allowed to be connected to output or inout module ports.
- **Evaluation:** The operands inside the replication are evaluated exactly once when the expression executes, even if the constant multiplier is zero.

Common Applications

- **Initialization:** It is frequently used to quickly construct wide bit patterns for initializing registers, lookup tables, or memory arrays. For instance, assigning {16{1'b0}} elegantly initializes a 16-bit counter to all zeros.
- **Sign Extension:** When performing arithmetic on signed numbers of varying bit-widths, the replication operator is the standard way to pad the most significant bits. If you need to extend a 4-bit negative number into an 8-bit signed number, you can replicate its sign bit to fill the extra space: { {8{1'b1}}, input_num }.
- **Nested Expressions:** Replication expressions can be freely nested inside standard concatenation expressions alongside other variables and sized constants (e.g., {a, b, 3'b000, {2{a}} }).
- **Bit-Swapping and Merging:** Concatenation is considered a best practice for performing bit-swapping or merging different buses, as it is much cleaner and more readable than writing multiple procedural assignment statements for individual bit ranges.

Nested Replication:

In Verilog, **nested replication** refers to the practice of placing a replication expression inside a regular concatenation expression.
This feature allows you to build complex, multi-bit vectors by seamlessly combining standard variables, fixed-sized constants, and dynamically replicated bit patterns all within a single overarching set of braces { }.

```
reg [1:0] a = 2'b10;  // 2-bit variable (Decimal 2)
reg [2:0] b = 3'b100; // 3-bit variable (Decimal 4)

// A single concatenation containing nested replications:
module des;
 reg [1:0] a;
 reg [2:0] b;

 initial begin
```

```
 a <= 2;
 b <= 4;

 #10;
 $display("a=%b b=%b res=%b", a, b, {a, b, 3'b000, {{2{a}}, {3{b}}}});
 end

endmodule
```

output:
xcelium> run
a=10 b=100 res=10100000101010010010 0
xmsim: *W,RNQUIE: Simulation is complete.

we use the replication operator **{16{1'b0}}** to create a 16-bit bit pattern consisting of 16 zeros (1'b0). This initializes the counter register to 0 at the start of the simulation.

Verilog Sign Extension:

A method used to extend a signed number with fewer bits to a wider bit-width by replicating its most significant bit (the sign bit), necessary when you need to perform arithmetic or logical operations on numbers that have different bit widths.

For example, if you have a 4-bit two's complement number representing -3 (which is 1101) and you want to mathematically add it to an 8-bit number, you must first sign-extend the 4-bit number to 8 bits (11111101) to make them compatible and preserve the correct negative value.
Here is how sign extension is handled in your Verilog designs:

1. Implicit Sign Extension (The signed Keyword) By default, standard wire and reg data types are treated as unsigned. If you perform arithmetic on these default types, Verilog will simply zero-extend the shorter operand to match the longer one, which will corrupt the value of negative numbers. To ensure the simulator automatically performs sign extension, you must explicitly declare the variables using the signed keyword.

Example:

```
module sign_extension(input signed [3:0] input_num,
 output reg signed [7:0] output_num);

 always @(*) begin
 if (input_num[3] == 1'b1) begin // if the sign bit is 1
 output_num = { {8{1'b1}}, input_num }; // extend with ones
 end else begin // if the sign bit is 0
 output_num = { {8{1'b0}}, input_num }; // extend with zeros
 end
 end
```

```
endmodule
```

2. Manual Sign Extension using Concatenation As we discussed in our previous conversation about concatenation, it is often a best practice to manually handle sign extension to make your architectural intent explicitly clear. You can combine the concatenation { } and replication {{ }} operators to look at the sign bit and pad the new upper bits with either 1s or 0s.

For example, to manually sign-extend a 4-bit input to an 8-bit output, you evaluate the 4th bit (the sign bit) and replicate it to fill the extra space:

```
always @(*) begin
   if (input_num == 1'b1) begin
      // If the sign bit is 1, replicate 1s to fill the new upper bits
      output_num = { {4{1'b1}}, input_num };
   end else begin
      // If the sign bit is 0, replicate 0s to fill the new upper bits
      output_num = { {4{1'b0}}, input_num };
   end
end
```

3. Arithmetic Shift Operators When scaling numbers down (such as dividing a value by a power of two using a right shift), you must be careful to maintain the sign bit. Standard logical shift operators (>>) will simply fill the vacant bit positions with zeros, which ruins signed arithmetic. Instead, you must use the **arithmetic right shift operator (>>>)**, which automatically preserves and replicates the sign bit as it shifts the vector.

1. ARITHMETIC OPERATORS

Operator	Symbol	Description	Operands	Result Type	Example
Addition	+	Adds two operands	1 or 2	Integer	a + b
Subtraction	–	Subtracts right operand from left	1 or 2	Integer	a – b
Unary Plus	+	Returns the value itself (positive)	1	Integer	+a
Unary Minus	–	Negates the value	1	Integer	–a
Multiplication	*	Multiplies two operands	2	Integer	a * b
Division	/	Divides left operand by right	2	Integer	a / b
Modulus	%	Remainder of division	2	Integer	a % b

2. RELATIONAL OPERATORS

Operator	Symbol	Description	Operands	Result Type	Example
Less than	<	True if left < right	2	1-bit (0 or 1)	a < b
Less than or equal	<=	True if left <= right	2	1-bit (0 or 1)	a <= b
Greater than	>	True if left > right	2	1-bit (0 or 1)	a > b
Greater than or equal	>=	True if left >= right	2	1-bit (0 or 1)	a >= b

3. EQUALITY OPERATORS

Operator	Symbol	Description	Operands	Result Type	Example
Logical equal	==	True if both operands are equal	2	1-bit (0 or 1)	a == b
Logical not equal	!=	True if operands are not equal	2	1-bit (0 or 1)	a != b
Case equal	===	True if both operands are equal (bit-for-bit, including X and Z)	2	1-bit (0 or 1)	a === b
Case not equal	!==	True if operands are not equal (bit-for-bit, including X and Z)	2	1-bit (0 or 1)	a !== b

4. LOGICAL OPERATORS

Operator	Symbol	Description	Operands	Result Type	Example
Logical AND	&&	True if both operands are true (non-zero)	2	1-bit (0 or 1)	a && b
Logical OR	||	True if any operand is true (non-zero)	2	1-bit (0 or 1)	a || b
Logical NOT	!	Inverts the operand	1	1-bit (0 or 1)	!a

5. SHIFT OPERATORS

Operator	Symbol	Description	Operands	Result Type	Example
Logical left shift	<<	Shift left, fill with 0	2	Same as left operand	a << 2
Logical right shift	>>	Shift right, fill with 0	2	Same as left operand	a >> 2
Arithmetic left shift	<<<	Shift left, sign bit extended	2	Same as left operand	a <<< 2
Arithmetic right shift	>>>	Shift right, sign bit extended	2	Same as left operand	a >>> 2

6. BITWISE OPERATORS

Operator	Symbol	Description	Operands	Result Type	Example
Bitwise AND	&	AND each bit of operands	2	Same as operands	a & b
Bitwise OR	|	OR each bit of operands	2	Same as operands	a | b
Bitwise XOR	^	XOR each bit of operands	2	Same as operands	a ^ b
Bitwise XNOR	~^	XNOR each bit of operands	2	Same as operands	a ~^ b
Bitwise NOT	~	Inverts each bit of operand	1	Same as operand	~a

7. UNARY OPERATORS

Operator	Symbol	Description	Operands	Result Type	Example
Unary plus	+	Returns the value itself (positive)	1	Same as operand	+a
Unary minus	–	Negates the value	1	Same as operand	–a
Logical NOT	!	Inverts logical value	1	1-bit (0 or 1)	!a
Bitwise NOT	~	Inverts every bit	1	Same as operand	~a
Reduction AND	&	AND all bits → 1 if all bits are 1	1	1-bit (0 or 1)	&a
Reduction NAND	~&	NAND all bits	1	1-bit (0 or 1)	~&a
Reduction OR	|	OR all bits → 1 if any bit is 1	1	1-bit (0 or 1)	|a
Reduction NOR	~|	NOR all bits	1	1-bit (0 or 1)	~|a
Reduction XOR	^	XOR all bits	1	1-bit (0 or 1)	^a
Reduction XNOR	~^	XNOR all bits	1	1-bit (0 or 1)	~^a

8. CONDITIONAL OPERATOR

Operator	Symbol	Description	Operands	Result Type	Example
Conditional (Ternary)	?:	If condition is true, returns expr1; else returns expr2	3	Same as expr1 and expr2	cond ? expr1 : expr2

9. CONCATENATION OPERATOR

Operator	Symbol	Description	Operands	Result Type	Example
Concatenation	{ }	Joins operands end to end	2 or more	Vector	{a, b, c}

10. REPLICATION OPERATOR

Operator	Symbol	Description	Operands	Result Type	Example
Replication	{N{a}}	Replicates operand a for N times	2	Vector	{4{a}}

11. STREAMING OPERATORS (SystemVerilog)

Operator	Symbol	Description	Operands	Result Type	Example
Stream left	{<<{a}}	Streams bits left to right	1	Vector	{<<{a}}
Stream right	{>>{a}}	Streams bits right to left	1	Vector	{>>{a}}

12. ASSIGNMENT OPERATORS

Operator	Symbol	Description	Operands	Result Type	Example
Blocking assignment	=	Assigns value immediately	2	—	a = b
Non-blocking assignment	<=	Assigns value at the end of time step	2	—	a <= b

13. COMPOUND ASSIGNMENT OPERATORS (SystemVerilog)

Operator	Symbol	Description	Operands	Result Type	Example
Add and assign	+=	a = a + b	2	—	a += b
Subtract and assign	–=	a = a – b	2	—	a –= b
Multiply and assign	*=	a = a * b	2	—	a *= b
Divide and assign	/=	a = a / b	2	—	a /= b
Modulus and assign	%=	a = a % b	2	—	a %= b
Left shift and assign	<<=	a = a << b	2	—	a <<= b
Right shift and assign	>>=	a = a >> b	2	—	a >>= b
AND and assign	&=	a = a & b	2	—	a &= b
OR and assign	|=	a = a | b	2	—	a |= b
XOR and assign	^=	a = a ^ b	2	—	a ^= b

NOTES:

- Operands can be constants, variables, nets, registers, expressions, or vectors.
- Result type depends on operands (Integer, Vector, or 1-bit).
- In expressions with mixed operators, precedence and associativity determine evaluation order.
- Use parentheses ( ) to override precedence.

PRECEDENCE (High → Low)

1. Unary 2. Power 3. *, /, % 4. +, - 5. <<, >>, <<<, >>>
6. Relational 7. Equality 8. Bitwise 9. Logical 10. Conditional 11. Assignment

ASSOCIATIVITY

- Most binary operators are Left-to-Right.
- Unary, Power, Conditional (?:), and Assignment are Right-to-Left.

DIFFERENCE BETWEEN ALL VERILOG OPERATOR TYPES

Operator Type	Purpose	Works On	Operands Required	Output Type	Key Point / Main Difference	Example	Example Result

1. Arithmetic Operators	Perform mathematical calculations	Numbers / Binary values	2 (binary) or 1 (unary +, -)	Numeric (same width or extended)	Performs actual math operations. Result can be multi-bit.	a = 4;
						b = 3;
						a + b
							7 (3'b111)

2. Relational Operators	Compare magnitudes (>, <, >=, <=)	Numbers / Binary values	2	1-bit (0 or 1)	Compares values and gives TRUE (1) or FALSE (0).	a = 4;
						b = 3;
						a > b
							1

3. Equality Operators	Check equality or inequality	Any data type (bits)	2	1-bit (0 or 1)	Checks if operands are exactly equal (==, ===) or not (!=, !==). ===/!== are case equality.	a = 4'b1010;
						b = 4'b1010;
						a == b
							1

4. Logical Operators	Evaluate logical conditions	Any expression (TRUE/FALSE)	1 (!) or 2 (&&, ||)	1-bit (0 or 1)	Works on whole expressions. Result always 1-bit.	a = 4'b0000;
						b = 4'b1010;
						a && b
							0

5. Bitwise Operators	Operate on each bit individually	Vectors (bits)	1 (~) or 2 (&, |, ^, ~^, ~&)	Same width as operands	Performs operation bit-by-bit. Result is a vector.	a = 4'b1010;
						b = 4'b1100;
						a & b
							4'b1000

6. Shift Operators	Shift bits left or right	Vectors (bits)	2	Same width as operands	Moves bits left/right. Vacated bits are filled with 0 (logical) or sign bit (arithmetic for <<<, >>>).	a = 4'b1011; a << 1	4'b0110
7. Reduction Operators	Reduce all bits of a vector to a single bit	Single vector	1	1-bit (0 or 1)	Combines all bits using the operator and gives a single 1-bit result.	a = 4'b1011; &a	0 (1&0&1&1 = 0)
8. Unary Operators	Operate on a single operand	Any data type	1	Same as operand	Needs only one operand. Example: +, -, ~, !, & (reduction) etc.	a = 4'b0101; ~a	4'b1010
9. Conditional (Ternary) Operator	Decision making (if-else in one line)	Condition + 2 expressions	3	Same as selected expression	If condition is true → first expression else → second expression.	sel = 1; y = sel ? 4 : 2;	4

10. Concatenation Operator	Join multiple signals into a larger vector	Vectors / Bits	2 or more	Vector (sum of all widths)	Joins operands end-to-end.	a = 4'b1010;
						b = 2'b11;
						{a, b}
							6'b101011

11. Replication Operator	Repeat a bit or vector multiple times	Bit/Vector + count	2	Vector	Replicates the operand N times.	a = 2'b10; {4{a}}	8'b10101010

12. Assignment Operators	Assign values to variables/signals	Variables / Nets / Registers	2	Assigned value	Used to store or update values. Blocking (=) or Non-blocking (<=).	a = 5; // blocking
						b <= 10; // non-blocking
							a = 5, b = 10

13. Compound Assignment Operators (SV)	Perform operation and assign in one step	Variables / Nets	2	Assigned value	Shorthand for operation + assignment. Example: a += b → a = a + b	a = 5; a += 3;	a = 8
14. Comma Operator (SystemVerilog)	Evaluate expressions sequentially	Expressions	2 or more	Last expression value	Evaluates left to right and returns the last expression value.	x = (a = 2, b = 3, a + b);	5

Quick Summary of Key Differences

Arithmetic	Relational	Equality	Logical	Bitwise	Shift	Reduction	Unary	Conditional	Concat/Replicate	Assignment
Does calculations, result can be multi-bit.	Compares magnitude, result is 1-bit.	Checks exact match / mismatch.	Works on conditions, result is 1-bit.	Operates on each bit individually.	Moves bits left or right, fills with 0 or sign.	Reduces all bits to single bit.	Works on a single operand.	Selects one of two expressions.	Builds larger vectors by joining or repeating.	Stores value in variable or register.